# آشنایی با ساختار برنامه‌های پایتون

پیش از آنکه برنامه‌های بزرگ‌تری بسازیم، لازم است بدانیم یک برنامه پایتون چگونه نوشته و سازمان‌دهی می‌شود. در این بخش با اجزای اصلی یک برنامه آشنا می‌شویم تا بتوانیم کدها را بهتر بخوانیم، بنویسیم و بررسی کنیم.

## ۱. فایل پایتون چیست؟

کدهای پایتون معمولاً در فایل‌هایی با پسوند `.py` ذخیره می‌شوند. برای مثال، فایلی با نام `main.py` می‌تواند شامل دستورهای یک برنامه باشد.

یک برنامه ساده را ببینید:

```
print("Hello, Python!")
print("I am learning to build software.")
```

تابع `print()` متنی را نمایش می‌دهد. وقتی این برنامه اجرا شود، دستورها به‌ترتیب اجرا می‌شوند و نتیجه روی صفحه نمایش داده می‌شود.

## ۲. اجزای اصلی یک برنامه

با بزرگ‌تر شدن برنامه، معمولاً با چند بخش اصلی روبه‌رو می‌شویم.

### وارد کردن کتابخانه‌ها

گاهی به قابلیت‌هایی نیاز داریم که در کتابخانه‌های آماده پایتون وجود دارند. با دستور `import` می‌توانیم آن‌ها را وارد برنامه کنیم.

```
import random

number = random.randint(1, 10)
print(number)
```

در این مثال، از کتابخانه `random` برای تولید یک عدد تصادفی بین ۱ تا ۱۰ استفاده کرده‌ایم.

### متغیرها و داده‌ها

متغیرها برای نگهداری داده‌ها به کار می‌روند؛ مانند نام کاربر، سن یا مبلغ یک خرید.

```
user_name = "Sara"
age = 20

print(user_name)
print(age)
```

نام متغیر بهتر است روشن و متناسب با کاربرد آن باشد تا خواندن و درک کد آسان‌تر شود.

### توابع

وقتی قرار است مجموعه‌ای از دستورها را چند بار استفاده کنیم یا بخشی از برنامه وظیفه مشخصی داشته باشد، می‌توانیم آن را در قالب یک تابع بنویسیم.

```
def greet_user(name):
    print(f"Hello, {name}!")


greet_user("Sara")
greet_user("Ali")
```

در این مثال، تابع `greet_user()` نام یک کاربر را دریافت می‌کند و به او خوشامد می‌گوید.

توضیحات یا کامنت‌ها

با استفاده از علامت # می‌توانیم توضیحاتی برای خواننده کد بنویسیم. پایتون این توضیحات را به‌عنوان دستور اجرایی در نظر نمی‌گیرد.

```
# Calculate the total price
price = 100
tax = 10

total = price + tax
print(total)
```

کامنت‌ها زمانی مفیدند که به درک دلیل یا هدف یک بخش از کد کمک کنند. بهتر است به‌جای توضیح واضحات، نکات مهم و تصمیم‌های برنامه را توضیح دهیم.

## ۳. ترتیب اجرای دستورها

در حالت معمول، پایتون دستورها را از بالا به پایین اجرا می‌کند؛ البته شرط‌ها، حلقه‌ها و فراخوانی توابع می‌توانند مسیر اجرای برنامه را تغییر دهند.

برای مثال:

```
first_number = 10
second_number = 5

result = first_number + second_number

print(result)
```

ابتدا دو متغیر ساخته می‌شوند، سپس مجموع آن‌ها در `result` قرار می‌گیرد و در پایان نتیجه نمایش داده می‌شود.

این ترتیب اهمیت دارد؛ چون معمولاً باید داده‌ها پیش از استفاده تعریف شده باشند.

## ۴. یک نمونه با ساختار منظم‌تر

در برنامه‌های بزرگ‌تر، بهتر است هر بخش وظیفه مشخصی داشته باشد. برای نمونه:

```
def calculate_total(price, tax):
    return price + tax


def main():
    price = 100
    tax = 10

    total = calculate_total(price, tax)
    print(f"Total: {total}")


if __name__ == "__main__":
    main()
```

در این برنامه:

- تابع `calculate_total()` محاسبه مبلغ نهایی را انجام می‌دهد.
- تابع `main()` مراحل اصلی برنامه را سازمان‌دهی می‌کند.

- عبارت `if __name__ == "__main__":` باعث می‌شود تابع `main()` هنگام اجرای مستقیم این فایل فراخوانی شود، نه صرفاً به‌دلیل وارد کردن آن از یک فایل دیگر.

در برنامه‌های بسیار ساده، استفاده از این ساختار همیشه ضروری نیست؛ اما آشنایی با آن به ما کمک می‌کند با رشد برنامه، کدها را منظم‌تر نگه داریم.

## ۵. چرا شناخت ساختار برنامه در عصر هوش مصنوعی مهم است؟

ابزارهای هوش مصنوعی می‌توانند برای ما کد تولید کنند؛ اما تولید کد به‌تنهایی به معنای ساخت یک نرم‌افزار درست و قابل-اعتماد نیست.

هنگام بررسی کدی که هوش مصنوعی پیشنهاد کرده، باید بتوانیم تشخیص دهیم:

- برنامه از کجا شروع می‌شود و چه مراحلی را اجرا می‌کند؟
- هر تابع چه وظیفه‌ای دارد؟
- داده‌ها کجا تعریف و چگونه استفاده می‌شوند؟
- آیا بخش‌های مختلف کد با یکدیگر سازگارند؟

لازم نیست از همان ابتدا تمام جزئیات زبان پایتون را بدانیم. هدف این است که به‌تدریج بتوانیم ساختار کلی برنامه را تشخیص دهیم، کد را بخوانیم و درباره درست‌بودن آن سؤال‌های مناسبی مطرح کنیم.

## جمع‌بندی

یک برنامه پایتون می‌تواند شامل دستورهای اجرایی، متغیرها، کتابخانه‌ها، توابع و کامنت‌ها باشد. برنامه‌ها معمولاً از بالا به پایین اجرا می‌شوند و با بزرگ‌تر شدن آن‌ها، تقسیم کد به بخش‌های مشخص به خوانایی و نگهداری بهتر کمک می‌کند.

تمرین: برنامه‌ای بنویسید که دو عدد را در متغیرهای جداگانه نگه دارد، حاصل ضرب آن‌ها را محاسبه کند و نتیجه را نمایش دهد. سپس همین برنامه را با استفاده از یک تابع بازنویسی کنید.